A university recently deployed an online exam proctoring system powered by AI to monitor students during remote assessments. Within the first week, students report false cheating alerts triggered by normal movements, and some legitimate violations go undetected. Faculty members complain that reviewing flagged sessions is time-consuming and stressful. The IT team suspects issues with the computer vision model's accuracy and integration with the exam platform. As the project manager, you must address these concerns quickly to avoid academic disruption while ensuring the system becomes reliable for future exams. Identify the key issues and list three strategies that balance an urgent fix with a robust, long-term solution while maintaining fairness and trust among students and faculty.

Key Issues

1. False Alarms

- The AI is mistakenly flagging normal actions—like looking around, stretching, adjusting posture, or fixing a headphone—as suspicious.
- This creates stress and makes students feel unfairly judged.
- Too many false alerts also overload the system and waste faculty time.

2. Undetected Violations

- Some real cheating incidents (like using another device, whispering answers, or looking off-screen for long periods) are not being caught.
- This shows the AI model is not accurately identifying true cheating behavior.
- It leads to unfair exams because honest students may feel disadvantaged.

3. Time-Consuming Session Reviews

- Teachers must manually review long video clips for every alert.
- Most alerts are not actual cheating, so teachers spend hours checking unnecessary cases.
- This increases stress and reduces trust in the proctoring system.

4. Poor Integration Between Systems

- The AI is not working smoothly with the exam platform.
- Problems like video lag, missing frames, or incorrect event syncing cause wrong alerts.
- Technical problems reduce accuracy and make the system unreliable.

Three Strategies for Urgent Fix + Strong Long-Term Solution

Strategy 1: Immediate Stabilization and Fairness Measures (Short-Term Fix)

- Temporarily reduce the AI's sensitivity so normal movements do not trigger alerts.
- Pause AI-based decisions for high-stakes exams and rely on manual or light monitoring to avoid unfair grading.
- Communicate clearly with students and faculty to explain the issues and assure them that no one will be punished based on faulty alerts.
- Provide quick filters for teachers so they can skip obvious false alarms and save time.

Goal: Reduce stress, prevent unfair consequences, and bring stability while deeper fixes are in progress.

Strategy 2: Technical Diagnosis and Model Improvement (Medium-Term)

- Perform a **Root Cause Analysis (RCA)** to find what is causing false alerts and missed cheating—whether it's the model, datasets, camera settings, or integration issues.
- **Retrain or fine-tune the AI model** using real examples from your university's environment so it better understands natural student behavior.
- Check and fix **integration errors**, such as video syncing, API delays, or missing logs.
- Conduct **small pilot tests** to evaluate improvements before using them in real exams.

Goal: Improve accuracy so the system detects real cheating and avoids flagging harmless actions.

Strategy 3: Build a Reliable, User-Friendly System for the Future (Long-Term Solution)

- Develop a more **balanced AI system** that focuses on patterns rather than single movements, reducing false triggers.
- Create a **teacher-friendly review dashboard** with:
    - Short highlight clips
    - Summaries of suspicious patterns
    - Confidence scores
    - Easy ways to skip false positives
- Introduce a **human-in-the-loop process**, where AI only assists but final decisions are made by teachers.
- Improve overall system design, especially **platform-AI integration**, to avoid technical errors.
- Test the improved system during **low-risk quizzes** before full-scale use.

Goal: Build long-term trust by ensuring the system is accurate, fair, and easy for faculty to work with.

Final Summary

The university's AI exam proctoring system is causing problems because it is flagging normal student movements as cheating, missing real violations, and forcing teachers to spend too much time reviewing videos. These issues come from both the AI model and its poor connection with the exam platform.

As the project manager, the solution is to:

1. **Fix urgent issues quickly** by reducing sensitivity, pausing the tool for major exams, and reassuring students and faculty.
2. **Find the real cause and improve the AI** through proper analysis, retraining, and fixing integration problems.
3. **Build a better long-term system** that is more accurate, more fair, and easier for teachers to use.

Question) A retail company is developing a mobile app for online shopping. During the initiation phase, the approved scope includes basic features like product browsing, cart management, and secure payment. Midway through development, marketing executives request additional features such as personalized recommendations, live chat support, AR-based product previews, and loyalty program integration. These requests were not part of the original scope and are causing confusion among the development team.

The project budget and timeline start to exceed initial estimates, and team members are unclear about which features are officially approved and which are pending. Using the principles of Software Scope Management, identify the main scope-related issues in this project and explain how the project manager can apply the processes of Define Scope, Create WBS, and Control Scope to manage stakeholder expectations, prevent scope creep, and ensure the project stays within agreed boundaries.

A) Scope-Related Issues in the Project

1. **Scope Creep:** This is the biggest issue. Marketing added new features without approval**,** and the PM allowed them informally.
   Examples of scope creep: Personalized recommendations, Live chat, AR product previews, Loyalty program
   These were not in the original project scope. As a result: Work increased, Time increased, Cost increased, Team became confused

2. **Incomplete Requirements Collection:** The requirements in the beginning were not gathered properly, because marketing's expectations came later. This means the PM did NOT fully use proper requirement-gathering tools like: Interviews, Focus groups, Prototypes, Workshops. Because of this, important features were missed.

3. **No Clear Scope Statement:** A good scope statement explains: What is included. What is not included. But here, the team does not know: Which features are approved. Which are still being discussed. This caused misunderstandings.

4. **No Proper WBS (Work Breakdown Structure):** A WBS breaks the entire project into clear, small tasks. Since no WBS was made:
   There is no clear baseline. Developers cannot check if a new task belongs to the approved scope. New features got mixed with approved ones

5. **No Scope Control / No Change Control Process:** Marketing gave new demands directly to the team. There was: No change request form, No review, No Change Control Board (CCB), No re-estimation of time and cost. Therefore, changes were happening without proper approval.

B) How the PM Should Fix This Using Scope Management Processes

1. **Collect Requirements:** Before starting development, the PM should gather **complete and accurate requirements** from all stakeholders using these tools:
   - **Interviews:** Talk directly with: Marketing, Sales, Customers. Ask what features they expect from the app.
   - **Focus Groups:** Hold a meeting with the marketing team to ask: Which features are essential? Which features can wait? This prevents misunderstandings.
   - **Historical Records:** Check previous company apps or similar shopping apps to understand needed features.
   - **Facilitated Workshops:** Bring all departments into one meeting: Marketing, Development, Customer support, Finance. Discuss requirements together.
   - **Brainstorming + Mind Mapping:** Generate ideas and organize them into categories.
   - **Multicriteria Decision Analysis:** Rank features using criteria like: Cost, Benefit, Time, Complexity. This helps decide what should be included.
   - **Surveys:** Take feedback from actual customers about what features they want.
   - **Prototypes:** Create simple screens to show how the app will look. This prevents confusion and helps stakeholders "see" the product.
   - **Benchmarking:** Compare features with big apps like Daraz or Amazon.

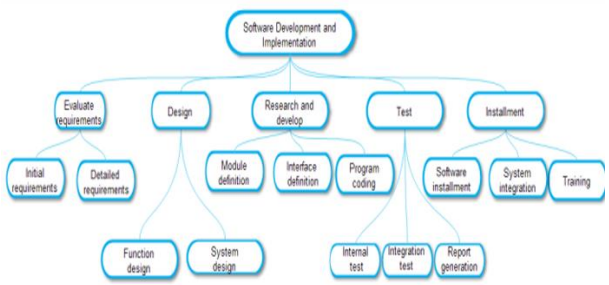
- **Document Analysis:** Analyze business plans, marketing strategy, and use cases.

Using these tools ensures **all important requirements are discovered early**, preventing late surprises.

2. **Define Scope — Make a Clear Scope Statement:** After collecting requirements, the PM must create a **Scope Statement** that clearly says:
   - **What is included (approved features):** Product browsing, Cart management, Secure payment.
   - **What is NOT included:** AR product preview, Recommendations, Loyalty program, Live chat

   This helps avoid confusion later because the team knows exactly what they must build.
   The scope statement also includes: Deliverables, Constraints, Assumptions, Acceptance criteria
   This becomes the official boundary of the project.

3. **Create WBS — Break the Approved Scope into Small Tasks:** The PM must create a Work Breakdown Structure**,** which breaks the whole project into small, manageable parts.
   Example WBS:
   1. Mobile App: 1.1 Product Browsing, 1.2 Cart Management, 1.3 Payment System, 1.4 UI/UX Screens
   2. Backend: 2.1 Database Setup, 2.2 API Development
   3. Testing: 3.1 Functional Testing, 3.2 Security Testing
   4. Deployment: 4.1 Play Store Release, 4.2 App Store Release



   If someone requests a new feature (e.g., AR preview), but it is not in the WBS**,** it is officially out of scope**.**
   The WBS becomes the project's baseline**.**

4. **Control Scope — Stop Scope Creep with a Formal Process:** When marketing requests new features, the PM must follow a formal change control system**.**
   Steps:
   1. Someone requests a new feature
   2. PM creates a **Change Request Form**
   3. Change Control Board (CCB) reviews it
      - How much extra time needed?
      - How much extra budget needed?
      - Does it add risks?
   4. If approved → update WBS, schedule, and budget
   5. If NOT approved → move feature to future release

   This ensures: No unauthorized changes, No confusion, Budget and timeline stay under control, Stakeholders agree on what will be built

Final Summary
Main Issues:

- Scope creep
- Missing requirements
- No clear scope
- No WBS
- No scope control

How PM fixes it:

- Use proper requirement-gathering tools (interviews, focus groups, prototypes, workshops).
- Create a detailed scope statement showing what is included and excluded.
- Make a WBS to break the project into clear tasks.
- Use Control Scope with a change request and CCB to stop scope creep.

This keeps the project **within time, cost, scope**, and prevents confusion among team members.